High-Level Data Design Document

Firefly III Intelligent Financial Insights

MLOps Project - Data Team Member

Chameleon Cloud (KVM@TACC)

1. Overview

This document describes the data infrastructure we built for the Firefly III ML system. Firefly III is an open-source personal finance manager, and our goal is to add two machine learning capabilities on top of it: real-time transaction categorization using a DistilBERT model, and monthly spending anomaly detection using XGBoost. The idea is straightforward when a user enters a transaction like, our system should automatically suggest that it belongs to the "Food at home" category, saving the user from manually tagging every purchase.

All infrastructure runs on Chameleon Cloud at KVM@TACC. I have used MinIO as our object storage layer (essentially an S3-compatible store running as a Docker container), and every pipeline component is containerized with Docker. The current deployment is accessible at IP 129.114.27.165, with MinIO's web console on port 9001 and the online feature service on port 8080.

The training data comes from the Bureau of Labor Statistics (BLS) Consumer Expenditure Survey, which provides detailed household spending records across multiple years (2020–2024). We transform this raw government survey data into realistic-looking transaction records that mimic what a Firefly III user would enter, complete with synthetic merchant names and regional variations.

2. Data Repositories

We use four MinIO buckets to organize data by its role in the pipeline, plus a PostgreSQL database that belongs to the Firefly III application itself. Each bucket serves a distinct purpose, and the separation makes it easy to reason about what data lives where and who is responsible for writing it.

2.1 Raw data (MinIO Bucket)

Purpose: This bucket stores the original, untouched data exactly as it is taken from external sources. We never modify anything in this bucket after the initial upload. If something goes wrong downstream, we can always trace back to exactly what raw data entered the system.

What is stored: BLS Consumer Expenditure Survey PUMD (Public Use Microdata) zip files for years 2020 through 2024, the CE-HG hierarchical grouping text files that map UCC codes to

human-readable item categories, and a lineage JSON file for each ingestion run that records provenance metadata.

Schema: The raw data consists of ZIP archives containing CSV files (MTBI for expenditure items, FMLI for household demographics). The lineage JSON follows our custom schema with fields: pipeline, version, timestamp, source, source_url, ucc_codes_mapped, and a years array with per-year statistics (mtbi_rows, parquet_key, size_mb).

Who writes: ingest_bls_data.py is the only process that writes to this bucket. It runs as a one-shot pipeline job.

When: On-demand, whenever we need to refresh the raw data from BLS. In practice, BLS releases new survey data annually, so this runs roughly once a year or when we first set up the system.

Versioning: Every file gets a timestamp-based name (e.g., bls_2024_20260405_173325.parquet). The lineage JSON records SHA-256 hashes of the raw data plus the exact BLS source URL, so we can verify data integrity and trace every bit of data back to its origin.

2.2 Processed data (MinIO Bucket)

Purpose: This is where data goes after initial cleaning and transformation, but before it becomes training ready. Think of it as the staging area — data here has been parsed from raw CSVs, joined with UCC mappings, and converted to an efficient columnar format, but it has not yet been shaped into the specific format that our ML models expect.

What is stored: Per-year Parquet files (bls_2020 through bls_2024) containing processed expenditure records, plus synthetic transaction parquet files generated from the BLS data.

Schema: Each parquet file has columns: NEWID (household ID), SEQNO, ALCNO, EXPNAME (item description), COST_ (expenditure amount), REF_MO/REF_YR (reference month and year), UCCSEQ (UCC sequence), SOURCE_YEAR, UCC_STR (UCC code string), ITEM_NAME (mapped from HG files), AGE_REF, FAM_SIZE, FINLWT21 (sampling weight), REGION, and FINCBTXM (income bracket).

Who writes: ingest_bls_data.py creates the initial per-year parquets. generate_synthetic_data.py adds the synthetic expansion files.

When: After each ingestion run. The synthetic data generation also runs as part of the data preparation pipeline.

Versioning: Filenames include both a timestamp and a data hash (e.g.bls_2024_20260405_173325.parquet). The synthetic data files also carry a version prefix and SHA-256 content hash in their filenames.

2.3 Training-data (MinIO Bucket)

Purpose: This bucket holds the final, model-ready datasets. Everything here is in the exact format that our DistilBERT categorization model and XGBoost trend model expect. This is also where versioned train/validation/test splits live, so we can always reproduce a specific training run.

What is stored: Two main training files under bls_pipeline/: categorization_training.parquet (currently 10,594 rows of labeled transactions) and trend_training.parquet. Under the versioned/ prefix, we store date-stamped directories (e.g., versioned/v20260405/) containing train, validation, and test split parquets plus a manifest JSON.

Schema: The categorization parquet has columns: description (merchant text), categories (multi-label list), amount (dollar value), persona_id, period, region, fam_size, and additional metadata. The versioned splits add a primary_category column derived from the multi-label field. The manifest JSON records pipeline version, split ratios, row counts per split, category distributions, candidate selection rules, and data hashes.

Who writes: data_pipeline.py creates the main training files. batch_training_pipeline.py creates the versioned splits.

When: data_pipeline.py runs after ingestion is complete. batch_training_pipeline.py runs on a weekly retraining cycle (or on-demand).

Versioning: Versioned directories follow the pattern versioned/v{YYYYMMDD}/. Each split file includes a content hash in its filename (e.g., train_00b2ca6b3290115d.parquet). The manifest JSON provides complete provenance for the entire version.

2.4 production-logs (MinIO Bucket)

Purpose: This bucket captures every prediction our model makes in production, along with the input transaction and confidence scores. This serves two purposes: first, it provides an audit trail so users can understand why a particular category was suggested; second, it feeds back into the retraining loop — once users correct or confirm predictions, those labeled examples become future training data.

What is stored: Individual JSON files, one per prediction request, organized by date.

Schema: Each JSON record contains transaction_id (UUID), description, amount, currency, date, true_category (ground truth if available), predicted_category, confidence (model probability), feedback (user correction, null until provided), and logged_at (ISO timestamp).

Who writes: data_generator.py (during simulation/testing) and the production inference service (in deployment). The online_feature_service.py handles the feature computation that precedes each prediction.

When: Every time a transaction is processed for categorization essentially in real-time, one JSON file per request.

Versioning: Files are partitioned by date in the path: requests/YYYY/MM/DD/{uuid}.json. This makes it straightforward to query logs for a specific day and also makes batch extraction simple for retraining.

2.5 PostgreSQL (Firefly III Application Database)

Purpose: This is the core application database for Firefly III itself. We do not directly manage this database, it is owned by the Firefly III application. However, it is important in the overall data design because it is the ultimate source of user transactions, and our ML system needs to integrate with it.

What is stored: User accounts, transactions, categories, budgets, tags, and all other Firefly III application data.

Schema: Standard Firefly III relational schema (normalized tables for accounts, transactions, categories, etc.).

Who writes: The Firefly III application server, based on user actions through the web UI or API.

Versioning: Managed by the Firefly III application. Not versioned by our ML pipeline.

3. Data Flow

The data flows through our system in two main paths: the offline training path (where we prepare data for model training) and the online inference path (where we compute features in real-time for predictions). Here is how each path works in practice.

3.1 Offline Training Path

The offline path starts with raw BLS data and ends with versioned, split training datasets ready for model consumption. There are four main stages:

Stage 1: External Data Ingestion

The BLS Consumer Expenditure Survey data is our primary external source. The ingest_bls_data.py script handles the complete ingestion process. It processes interview survey zip files (intrvw20.zip through intrvw24.zip) by extracting MTBI (monthly expenditure) and FMLI (household characteristics) CSV files from each archive. For each year, it parses the CE-HG hierarchical grouping files to build a mapping from six-digit UCC codes to human-readable item names (there are approximately 867 unique UCC codes across all years). The script then joins expenditure records with household demographics and UCC descriptions, producing one Parquet

file per year. Each file is uploaded to the processed-data bucket in MinIO, and a lineage JSON recording the complete provenance is saved to raw-data.

Stage 2: Data Transformation

The data_pipeline.py script takes the processed parquets and transforms them into realistic transaction records suitable for training a categorization model. This is a six-step process:

Step 1: Persona Sampling: We sample 750 representative consumer personas from the FMLI household data, weighted by FINLWT21 (the BLS sampling weight), so that our training data reflects the actual distribution of American households rather than overrepresenting any particular demographic.

Step 2: Item-to-Transaction Aggregation: Raw BLS records describe individual expenditure items (e.g., "Eggs" or "Gasoline"). We aggregate these into transaction-level records that look like what a user would actually see on their bank statement.

Step 3: Multi-Label Category Assignment: Each transaction is assigned one or more categories based on its UCC hierarchy and semantic similarity. Most transactions receive 2–3 category labels, reflecting the reality that a purchase at Target could reasonably be categorized as "Household" or "Clothing" depending on what was bought.

Step 4: Synthetic Merchant Name Generation: To make the data realistic, we generate merchant names with regional variations. A grocery purchase might become "KROGER #1247 SPRINGFIELD IL" or "SAFEWAY #892 PHOENIX AZ" depending on the household's region.

Step 5: Quality Validation: The pipeline runs automated checks on category distribution, amount statistics, and multi-label coverage before proceeding.

Step 6: Build Training Datasets: The final categorization_training.parquet (10,594 rows across 9 categories) and trend_training.parquet are uploaded to the training-data bucket.

Stage 3: Synthetic Data Expansion

Since our dataset is under 5GB, we have expanded it through synthetic data generation. The generate_synthetic_data.py script loads a 10% sample from each year's processed parquet, maps expenditure items to realistic categories using both UCC codes and text-based keyword matching, generates synthetic merchant descriptions, and applies small random perturbations to amounts (0.9x to 1.1x). The resulting synthetic transactions are saved alongside the original data in the processed-data bucket, with full metadata including a content hash for traceability.

Stage 4: Batch Training Pipeline

The batch_training_pipeline.py runs on a weekly cycle (or on-demand for retraining). It loads the categorization training data from MinIO, applies candidate selection rules (have described in Section 5), performs a chronological train/validation/test split to prevent data leakage (have

described in Section 6), and uploads the versioned splits to training-data/versioned/v{YYYYMMDD}/. A manifest JSON accompanies each version with complete metadata for reproducibility.

3.2 Online Inference Path

When a user enters a new transaction in Firefly III, the real-time inference path kicks in. The online_feature_service.py runs as an HTTP server on port 8080, exposing a POST /compute-features endpoint. When it receives a transaction, it performs the following feature computation:

- Text preprocessing: The raw description is lowercased, long numeric sequences are replaced with a [NUM] token (to normalize store numbers and reference IDs), and special characters are cleaned up. For example, "KROGER #1247 SPRINGFIELD IL" becomes "kroger [NUM] springfield il".
- Amount features: The raw dollar amount is log-normalized using log1p to handle the wide range of transaction values (from $0.16 coffee to $50,000+ purchases) in a way that is more amenable to model training.
- Temporal features: The date is decomposed into month_of_year, day_of_week, and is_weekend, capturing spending patterns that vary by time (people spend differently on weekends, and certain categories are seasonal).
- User history features: The service maintains an in-memory rolling window of the last 100 transactions per user. From this, it computes the average past transaction amount and a deviation ratio (current amount divided by historical average), which helps the model flag unusual spending patterns.

The resulting 10-feature vector is then passed to the DistilBERT model for categorization inference. Predictions and confidence scores are logged to the production-logs bucket for future retraining.

3.3 Production Data Collection (Feedback Loop)

The data_generator.py simulates realistic production traffic by sending synthetic transactions to the inference endpoints. It draws from a pool of 20 representative transaction templates spanning all major spending categories, introduces random amount variation, and logs each prediction along with its confidence score and the ground-truth category to MinIO. In production, this feedback loop would be driven by actual user corrections when a user changes a predicted category, that correction is stored as a labeled training example that feeds into the next retraining cycle.

4. Data Flow Diagram

The diagram below illustrates the complete data flow through the system, from external sources through processing, training, and inference.

Figure 1: Data Flow Diagram

5. Versioning Strategy

We use a multi-layered versioning approach to ensure that every piece of training data can be traced back to its source and that any training run can be reproduced.

5.1 Filename-Based Versioning

Every file we produce includes a timestamp (YYYYMMDD_HHMMSS format) and a 16-character SHA-256 content hash in its filename. For example, bls_2024_20260405_173325.parquet tells us exactly when this file was created. This approach is simple but effective we can immediately tell from the filename alone whether two files are different versions of the same data.

5.2 Version Manifests

Each run of the batch training pipeline produces a manifest JSON file stored under versioned/v{YYYYMMDD}/. This manifest is the complete record of a training dataset version. It includes: the source data hash (so we know which upstream data was used), the pipeline version, the candidate selection rules that were applied, the leakage prevention strategy, row counts and category distributions for each split, and enough information to reproduce the exact same splits given the same source data.

5.3 Lineage Tracking

At the ingestion layer, every run of ingest_bls_data.py saves a lineage JSON to the raw-data bucket. This records the BLS source URL, the download timestamp, SHA-256 hashes of all raw files, the number of UCC codes successfully mapped, and per-year statistics (how many MTBI rows were processed, the output parquet key, and file sizes). If we need to debug a data quality issue, we can trace it all the way back to the specific BLS download that produced it.

6. Candidate Selection

Not every record in our processed data is suitable for training. The batch_training_pipeline.py applies a carefully designed set of candidate selection rules before creating training splits. These rules are documented in the manifest for each version.

First, we remove incomplete records: any row missing a description, amount, or category is dropped. Transactions with zero or negative amounts are also excluded, as they typically represent refunds or data entry errors that would confuse the categorization model.

Second, we enforce a minimum category count of 50 examples. Categories with fewer than 50 transactions are excluded from training because there simply is not enough data to learn a reliable

decision boundary. This is a practical trade-off we would rather have a model that is excellent at categorizing 7 common spending categories than one that tries (and fails) to handle 15 categories with sparse training data.

Third, we apply per-category outlier removal at the 99th percentile. For each category, any transaction with an amount above p99 for that category is removed. The rationale is that extreme amounts (e.g., a $619,000 "Miscellaneous" transaction) are likely data anomalies that would distort the model's understanding of typical spending patterns. After applying all these rules, our dataset goes from 10,594 to approximately 10,485 usable examples across the retained categories.

7. Leakage Prevention

Data leakage is one of the most common and damaging mistakes in applied ML, and we have taken several concrete steps to prevent it in our pipeline.

The most important measure is strictly chronological splitting. We never randomly shuffle data before splitting instead, we sort all records by date and take the oldest 70% as the training set, the next 15% as validation, and the newest 15% as the test set. This mirrors how the model will be used in production: it will always be predicting on transactions that are newer than its training data.

The pipeline includes explicit leakage checks. After splitting, it verifies that the maximum date in the training set is less than or equal to the minimum date in the validation set, and similarly for validation versus test. If either check fails, the pipeline halts with a "LEAKAGE DETECTED" error rather than silently producing a contaminated dataset.

In the online feature computation path, all features use only data available at prediction time. The rolling average amount is computed from past transactions only (the current transaction is excluded from the average). There is no future information leakage in any computed feature.

Finally, user feedback labels when a user corrects a predicted category are only used in retraining batches that occur after the prediction date. A correction made on April 5th will only appear in training data for models trained after April 5th.

8. MinIO Bucket URLs

All buckets are accessible via the MinIO web console for browsing and verification by course staff.

MinIO Console: http://129.114.27.165:9001

Username: firefly-access-key

Password: firefly-secret-key

Bucket URLs:

Bucket	URL
raw data	http://129.114.27.165:9001/browser/raw-data
processed data	http://129.114.27.165:9001/browser/processed-data
training-data	http://129.114.27.165:9001/browser/training-data
production-logs	http://129.114.27.165:9001/browser/production-logs